DATA STRUCTURES AND ALGORITHMS

القسم الاول

هياكل البيانات الخطية (Linear structures) :

هو نوع هياكل البيانات الذي يعتمد مبدا التسلسل في خزن عناصر الهيكل البياني في الذاكرة اي بمعنى ان العنصر الاول يتبعه العنصر الثاني والثاني يتبعه العنصر الثالث وهكذا بقية العناصر. تقسم هياكل البيانات الخطية الى قسمين: الاول هو هياكل البيانات الخطية الثابتة (**Static linear structures**) والثاني هو هياكل البيانات الخطية المتحركة (**Dynamic linear structures**) .

- **هياكل البيانات الخطية الثابتة (Static linear structures)**: هي هياكل البيانات التي تستخدم مبدأ تسلسل العناصر وحجز عناوين ذاكرة متجاورة لتمثيل العناصر داخل الهيكل البياني. مثال على ذلك المصفوفة .

المصفوفة Array : .هي عبارة عن مجموعة من المواقع الخزنية في الذاكرة تتصف بما يلي :

1- جميع المواقع تكون من نوع بياني واحد ،حسب صيغة التعريف .....float ,int ,char

2- يمكن الوصول عشوائيا (Randomly accessed) الى اي موقع من مواقع المصفوفة دون الاعتماد على اي موقع اخر في نفس المصفوفة

3- مقدار الوقت المطلوب للوصول الى اي موقع في المصفوفة هو مقدار ثابت ومتساوي لجميع العناصر .

4- مواقع عناصر المصفوفة تبقى ثابتة ولاتتغير اثناء التعامل مع اي من عناصر المصفوفة.

5- تمثل المصفوفة باستخدام مواقع متعاقبة في الذاكرة .

1- تمثيل المصفوفة الاحادية في الذاكرة 1D array representation in memory :

في لغة البرمجة تعرف المصفوفة كالاتي :

Int {*or any other type*} x [N]

Ex:

Int a[10]; float a[20];

وهذا يعني تعريف هيكل بياني يستوعب مجموعة من العناصر البيانية عددها N مثلا باسم بياني واحد مثال X ويستخدم الدليل [index] للوصول الى العنصر البياني المطلوب وتتراوح قيمة الدليل (1=<i<=n) وبموجب هذا التعريف يحدد المعالج مناطق ذاكرة متعاقبة لاستيعاب مجموعة العناصر البيانية للمصفوفة ويكون عنوان الذاكرة الاول مخصصا للعنصر الاول في المصفوفة وهو ما يطلق عليه عنوان البداية (Base address, BA). يستخدم معالج اللغة Compiler المعادلة التالية للوصول الى بيانات المصفوفة في الذاكرة :

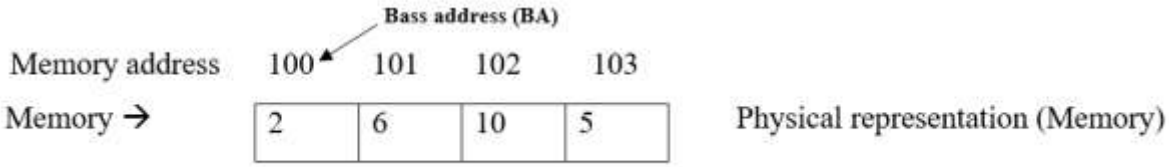
Location (x[i]) = Base Address + (i-1)	(1)

حيث i يمثل موقع العنصر المطلوب

Example:

a [4] = {2,6,10,5};

a[1]=2, a[2]=6, a[3]=10, a[4]=5 Logical representation (programmer)



لذلك لو فرضنا اننا نريد طباعة العنصر الثالث من عناصر المصفوفة اعلاه فان المعالج سيستخدم المعادلة رقم (1) للوصول الى العنصر المطلوب في الذاكرة وكما يلي

Loc (a[i]) = BA+ (i-1) → Loc (a[3])=100+(3-1) → Loc(a[3]) = 302

Ex 2: Let int x [n] any array. Compute the memory address of the element x [4] when the bass Address is 500?

Solution:

i=4

Location(x [4]) =500+ (4-1)

=500+3=503

* فعندما يتضمن البرنامج اية اشارة او تعامل مع عناصر المصفوفة في اي ايعاز مثل cin>>x[i] او cout<< x[i] او غيرها فان المترجم يعتمد المعادلة 1 المشار اليها اعلاه لتحديد الموقع المطلوب .

تمثيل المصفوفة الثنائية في الذاكرة Representation of 2D array in memory :

في لغة البرمجة تعرف المصفوفة الثنائية كالاتي ،

int {or any other type} A [M][N]

Ex:

Int A [5][10];

Float A [4][6];

وهذا يعني تعريف هيكل بياني اسمه (A) يستوعب مجموعة من العناصر البيانية عددها (M*N) ويستخدم دليلين للوصول الى العناصر البياني المطلوب وهما

1=<i<=M لتحديد الصف الذي فيه العنصر

1=<j<=N لتحديد العمود الذي فيه العنصر

فمثلا العنصر A[3][5] حيث i=3 وj=5

وهذا يعني ان العنصر يقع في الصف الثالث والعمود الخامس .

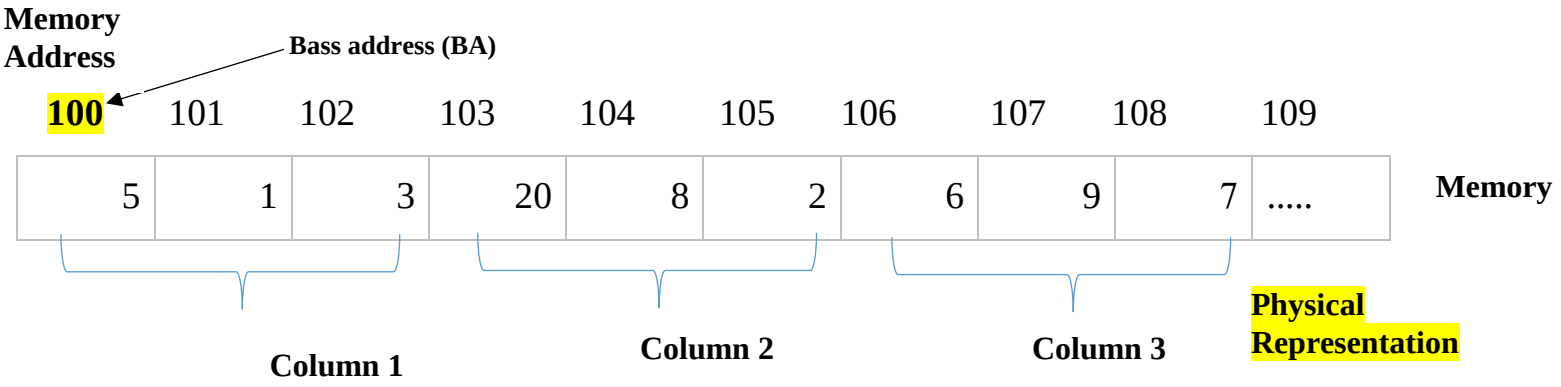
Ex: a[3][4] = $\begin{matrix} 5 & 20 & 6 & 5 \\ 1 & 8 & 9 & 1 \\ 3 & 2 & 7 & 4 \end{matrix}$ → = $\begin{matrix} a11 & a12 & a13 & a14 \\ a21 & a22 & a23 & a24 \\ a31 & a32 & a33 & a34 \end{matrix}$ → Logical representation

المثال اعلاه يعكس التمثيل المنطقي للمصفوفة الثنائية من وجهة نظر المبرمج. يعتمد مترجم اللغة (compiler) احدى الطريقتين الاتيتين لتمثيل المصفوفة الثنائية البعد فيزيائيا في الذاكرة وكما موضح ادناه .

1- طريقة الاعمدة Column method :

وهذه الطريقة مستخدمة في لغة الفورتران والبيسك.

توخذ عناصر العمود الاول (j=1) للمصفوفة وتخزن في مواقع متعاقبة في الذاكرة ثم يتم اخذ عناصر العمود الثاني ليتم خزنها بعد اخر عنصر من عناصر العمود الاول وهكذا وصولا الى العمود الاخير في المصفوفة كما موضح بالشكل ادناه.



وعليه فان احتساب موقع العنصر A[i][j] يكون وفق المعادلة التالية :

$$\text{Location}(A[i][j]) = \text{Base Address} + M \times (j-1) + (i-1) \qquad (2)$$

حيث ان M تمثل عدد الصفوف الكلي للمصفوفة

Ex: Let int s [3] [4] any Array, compute the location of the element [3] [2] when Base Address is (100).

Sol:

i=3, j=2, M=3, N=4

Location (s [3] [2]) =100+ 3 (2-1) + (3-1)

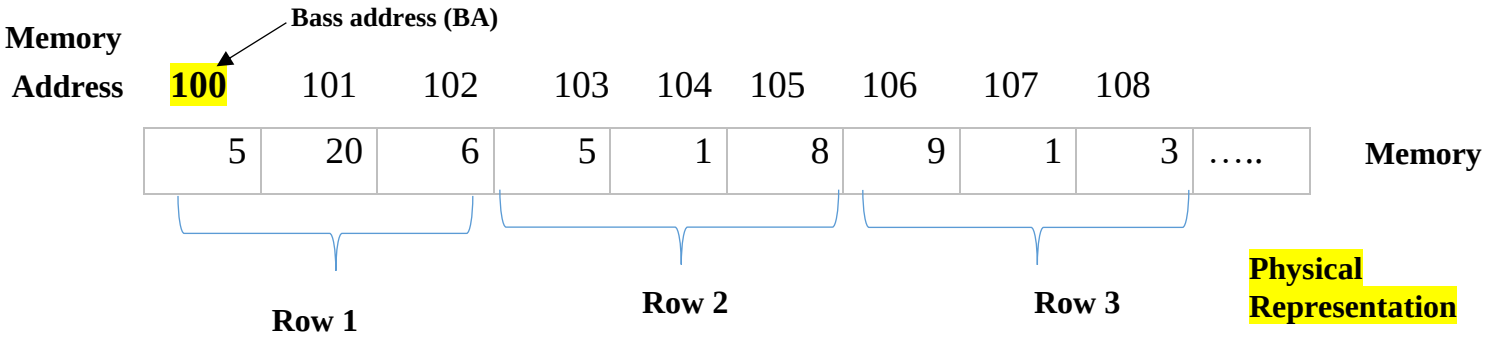
= 100+ 3+2 = **105**

2- طريقة الصفوف Row method:

وهذه الطريقة مستخدمة في لغة باسكال ،كوبول ، C++

حيث توخذ جميع عناصر الصف الاول للمصفوفة الثنائية وتخزن في الذاكرة ابتداء من موقع البداية (Base Address).

ثم توخذ جميع عناصر الصف الثاني للمصفوفة وتخزن في الذاكرة بعد اخر عنصر من عناصر الصف الاول وهكذا الى نهاية الصفوف.



ولهذا فان احتساب موقع العنصر A[i][j] يكون وفق المعادلة التالية :

Location (A[i] [j]) = Base Address + N * (i-1) + (j-1) (3)

حيث ان N تمثل عدد الاعمدة الكلي للمصفوفة

Ex: Let int A [3] [4]

Compute the location of the element A [2] [4] ?

When the base Address is 100.

Sol:

i=2, j=4 , M=3 , N =4

Location (A [2] [4]) = 100+ 4 * (2-1) + (4-1)

= 100 + 4 + 3 = **107**